

Sompo Risk MVP

Relatório técnico corrigido · Sprint 4

Demonstração acadêmica com dados e rótulos simulados. Resultados medidos nesta versão, sem alegação de desempenho em produção.

Integrantes

Nome informado	RM
Camila	569629
João	571052
Lucas	569580
Gabriel	569179

Sobrenomes, responsabilidades individuais, User Stories e histórico das Sprints 1-3 não foram fornecidos. Devem ser registrados pelo grupo antes da submissão.

1. Objetivo e fluxo

O MVP recebe eventos de telemetria, clima e operação; valida tipos, faixas e timestamp UTC; registra rejeições e duplicidades; persiste eventos e scores; calcula variáveis; produz alerta e disponibiliza API e dashboard. SQLite torna a execução local simples. Um identificador de evento assegura idempotência.

Entrada JSON → validação → SQLite e auditoria → features → regressão logística → score → alerta → API e dashboard. O código correspondente está nas pastas app/, data/, models/, dashboard/ e tests/.

2. Contrato, qualidade e limites

Campos obrigatórios: event_id, equipment_id, timestamp, region, operation_type, speed_kmh e maintenance_days. Velocidade aceita 0–180 km/h; temperatura –20–180 °C quando presente; chuva não negativa; tipo de operação vem de uma lista controlada. Temperatura ausente gera indicador e imputação explícita para a demonstração. Eventos inválidos são rejeitados com motivo.

O cadastro real de equipamentos, fontes externas, retries/circuit breaker, telemetria por horas de uso e proteção contra reescrita do banco não estão implementados. Os endpoints de relatório agregam os eventos disponíveis, sem extrapolar para taxa por mil horas.

3. Modelo e avaliação

A base sintética é gerada por data/generate_synthetic.py com semente 42 e 400 eventos. O rótulo de risco em 24 horas é artificial. A separação por tempo usa 280 registros para treino, 60 para validação e 60 para teste. O limiar é selecionado na validação; o teste é reservado à medição final. O modelo é regressão logística com padronização e imputação.

O limiar de classificação para a avaliação é distinto das faixas operacionais de exibição (0–39, 40–69, 70–100). Os fatores explicativos exibidos derivam de regras transparentes e não são atribuições causais do classificador.

4. Resultados efetivamente medidos

Métrica	Teste sintético
Recall	0.7200
Precisão	0.4865
F1	0.5806
PR-AUC	0.6675
Brier	0.2218
Limiar escolhido na validação	0.25

Esses resultados não atingem as metas ilustrativas do documento inicial. Eles não servem para demonstrar qualidade preditiva em frotas reais. O arquivo `models/risk_model.json` contém o hash da base e as métricas reproduzíveis.

5. Integração, segurança e evidências

Sete testes automatizados passaram nesta execução: contrato e rejeição, idempotência, alerta, API, controle de acesso demonstrativo e detecção de adulteração pontual do log. A ingestão de três eventos fictícios gerou scores 57, 2 e 29 com o modelo sintético versionado. Esses dados constam em `docs/evidencias/`.

A API diferencia operador, gestor e analista por tokens estáticos de demonstração. O log usa hashes encadeados e `correlation_id`. Isso permite detectar alteração isolada, mas não oferece proteção contra um administrador que reescreva toda a base. Para produção é necessário autenticação e autorização por equipamento, TLS e auditoria imutável externa.

6. Entrega e pendências externas

O pacote inclui código, banco gerado localmente, modelo sintético versionado, README, testes, relatório, evidências e roteiro de vídeo. Ainda dependem do grupo: histórico verdadeiro das Sprints 1-3 e User Stories, responsabilidades de cada integrante, captura do dashboard, gravação de vídeo com narração humana (até 5 minutos) e publicação não listada no YouTube, repositório GitHub privado com acesso a `fiap-tutoria` e integrantes.

Próximos passos: obter dados históricos autorizados, definir o evento de risco, validar calibração e desempenho por região/equipamento, medir impacto de falsos positivos e testar a solução em piloto supervisionado.

Documentação de execução completa: `README.md`.